



NTNU

Kunnskap for en bedre verden

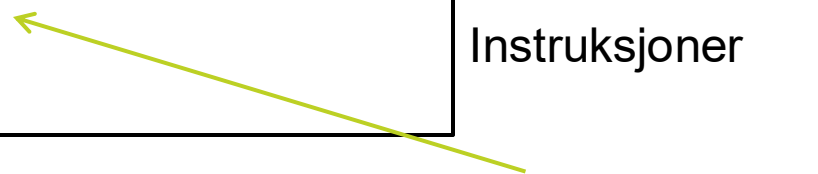


Forelesning 8: Farer, avbrudd og høyere ytelse

TDT4160 Datamaskiner

The diagram illustrates the MIPS architecture with the following components and data flow:

- PC (Program Counter):** Holds the current instruction address.
- Instruction Memory:** Provides instructions based on the PC address. It outputs instructions 19-15, 24-20, 23-19, and 24-0.
- Adder:** Takes the PC and an immediate value 'i' to calculate the next PC value.
- Register File:** Contains 32 registers. It provides Read data 1-4 and Write register data to the ALU and ALU2. It also receives Write register data from the ALU2.
- ALU (Arithmetic Logic Unit):** Performs operations on Read data 1-4 and Write register data. It outputs ALU data to the Register File and ALU2.
- ALU2:** Performs operations on ALU data and Write register data. It outputs ALU2 data to the Register File and the Memory Unit.
- Memory Unit:** Receives Address data and Write enable signals. It outputs Read data to the Register File and ALU2.
- Write Enable:** A control signal that enables writing to the Register File and Memory Unit.
- Read Enable:** A control signal that enables reading from the Register File and Memory Unit.
- Instruction Memory Address:** The address used to access the Instruction Memory.
- Read Data:** Data read from the Register File and Memory Unit.
- Write Register Data:** Data written to the Register File and Memory Unit.
- ALU Data:** Data output from the ALU.
- ALU2 Data:** Data output from the ALU2.
- Write Enable:** A control signal that enables writing to the Register File and Memory Unit.
- Read Enable:** A control signal that enables reading from the Register File and Memory Unit.



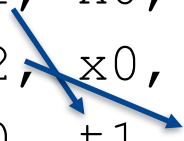
... og vi har begynt jobben med å lage raskere og bedre **samlebåndsprosessorer**

Farer («Hazards»)

- En avhengighet er en egenskap ved programmet
 - ... mens en fare er en avhengighet som påvirker utføringen av programmet på en gitt maskin
- Klasser av farer:
 - **Strukturfare («Structural hazard»):** En enhet instruksjonen trenger er ikke tilgjengelig
 - **Datafare («Data hazard»):** Instruksjonen kan ikke utføres fordi data ikke er tilgjengelig
 - **Kontrollfare («Control hazard»):** Vi vet ikke hvilken instruksjon som skal utføres

Dataavhengigheter

```
addi t1, x0, 17
addi t2, x0, 13
add s0, t1, t2
```



... som blir datafarer på 5-stegsmaskinen vår

Hvordan håndtere farer?

- Unngå ressurskonflikter
- VidereSending («Forwarding»)
- Stans («Stalling»)
- Prediksjon («Prediction»)





Tema 5.1: Samlebåndspresessor med 5 steg (Kapittel 4.8)

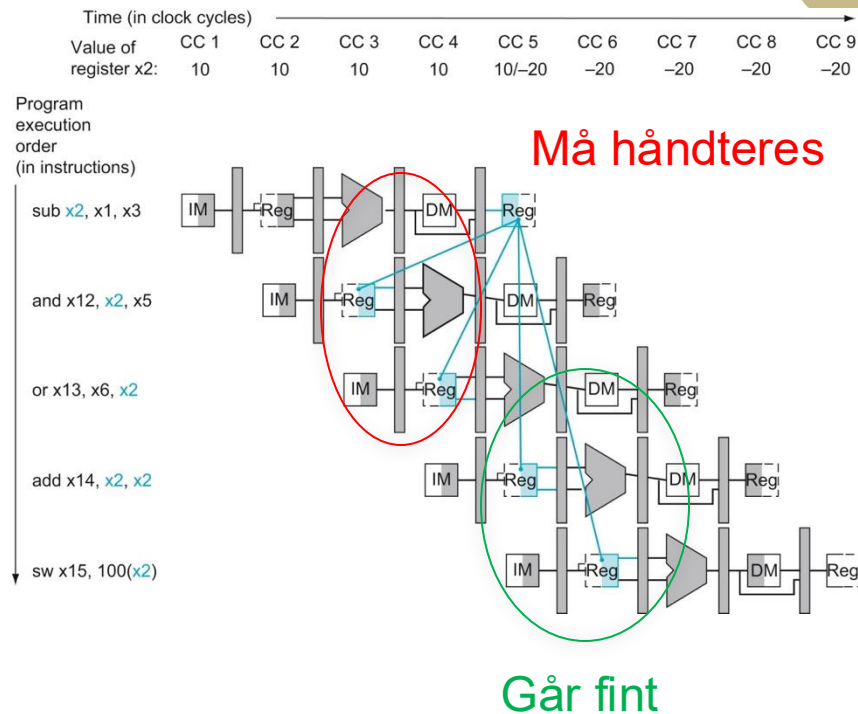
DATAFARER

Læringsutbytte T5.1

- Studenten skal kunne beskrive fordeler og ulemper ved samlebåndsprosessoren sammenliknet med flersykelprosessoren og enkeltsykelprosessoren.
- Studenten skal kunne forklare hvordan oppstartskostnad og balanse mellom steg i samlebåndet påvirker ytelse.
- Studenten skal kunne beskrive forskjellen mellom avhengigheter og farer samt gjengi de tre hovedgruppene av farer (strukturfarer, datafarer, og kontrollfarer).
- Studenten skal kjenne til de fire overordnede strategiene for å håndtere farer (unngåelse, videresending, stans, og prediksjon)
- Studenten skal kunne forklare den grunnleggende mikroarkitekturen til 5-stegs samlebåndsprosessoren (kunne gjengi og forklare figur 4.43).
- Studenten skal kunne forklare hvordan kontroll implementeres i 5-stegs samlebåndsprosessoren.
- **Studenten skal kunne forklare hvordan videresending og stans kan brukes til å håndtere datafarer samt kunne analysere hvordan strategiene påvirker prosessorens ytelse.**
- Studenten skal kunne forklare hvordan kontrollfarer kan håndteres med stans og prediksjon samt analysere hvordan strategiene påvirker prosessorens ytelse.

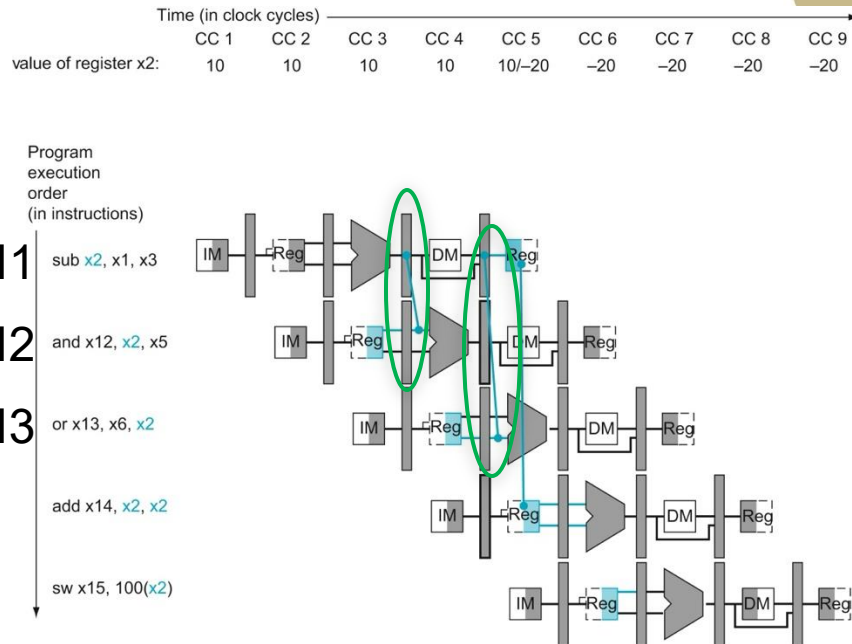
Hvordan håndtere datafarer?

- Vi må håndtere datafarer på en måte som:
 - Oppnår korrekt utføring av alle program
 - Minimerer tap av ytelse
- Datafarer oppnår når:
 - (i) En instruksjon trenger data produsert av en tidligere instruksjon (**avhengighet**)
 - (ii) Instruksjonene utføres nært nok i samlebåndet til at verdiene ikke kan hentes fra registerfila
- Å skrive til registrene i første halvdel av klokkesyklusen og lese i andre halvdel unngår en fare
 - Verdien produsert av `sub x2, x1, x3` er tilgjengelig i registerfila når `add x14, x2, x2` leser den



Videresending («Forwarding»)

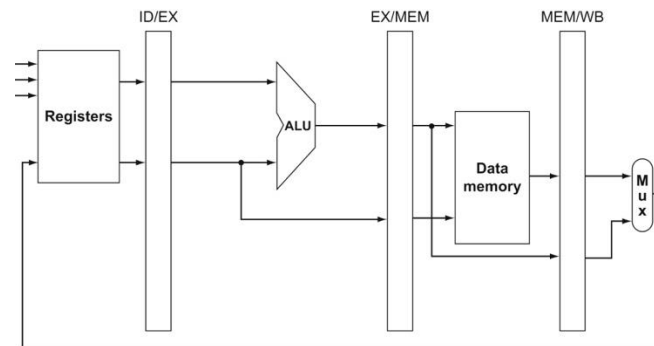
- Observasjon: Verdien en instruksjon trenger er ofte tilgjengelig i et samlebånds-register lenge før den skrives til registerfila
- Løsning: Videresending
 - Vi legger til maskinvare som bytter ut verdien lest fra registerfila med den oppdaterte verdien
 - I2: Rett verdi ligger i EX/MEM
 - I3: Rett verdi ligger i MEM/WB



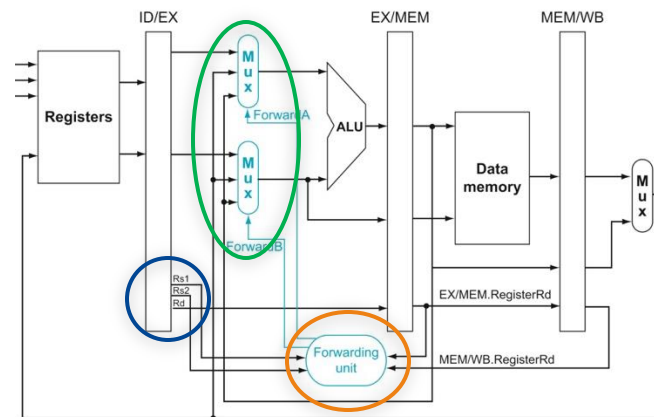
Merk at verdiene videresendes fra venstre mot høyre → **altså fremover i tid**

Implementere videresending

- Vi må lagre registernumrene i samlebåndsregistrene
 - ... fordi nå «trenger vi dem senere»
- Vi må legge logikk som velger den riktige verdien
 - Videresendingsenhet («Forwarding unit»)
- Vi legger til ekstra multiplekserene på ALU-inngangene slik at de kan ta imot verdier fra EX/MEM og MEM/WB



a. No forwarding



b. With forwarding

Eksempel: Videresending

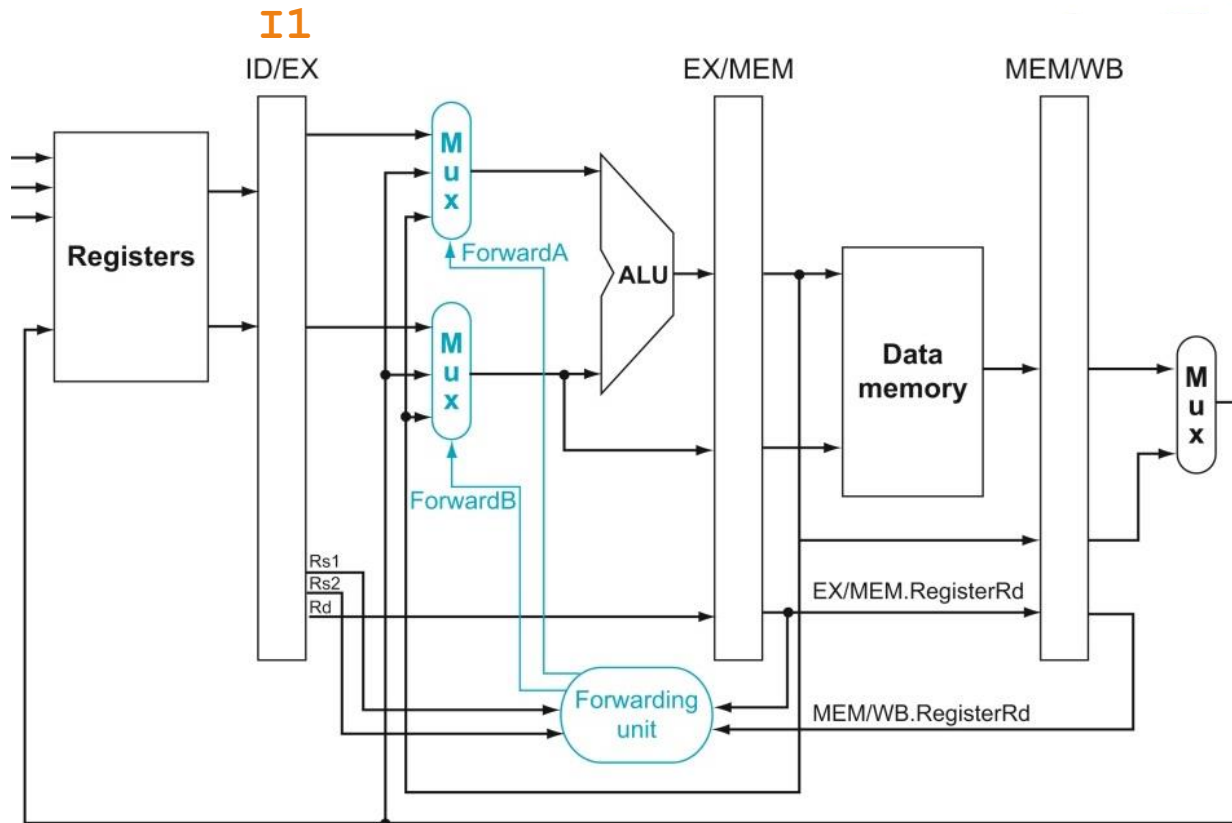
x1	5
x2	10
x3	3
x4	4
x5	5

I1: add **x3**, x1, x2

I2: sub x4, **x3**, x2

I3: mul x5, x2, **x3**

Slutten av
klokkesykel 1



Eksempel: Videresending

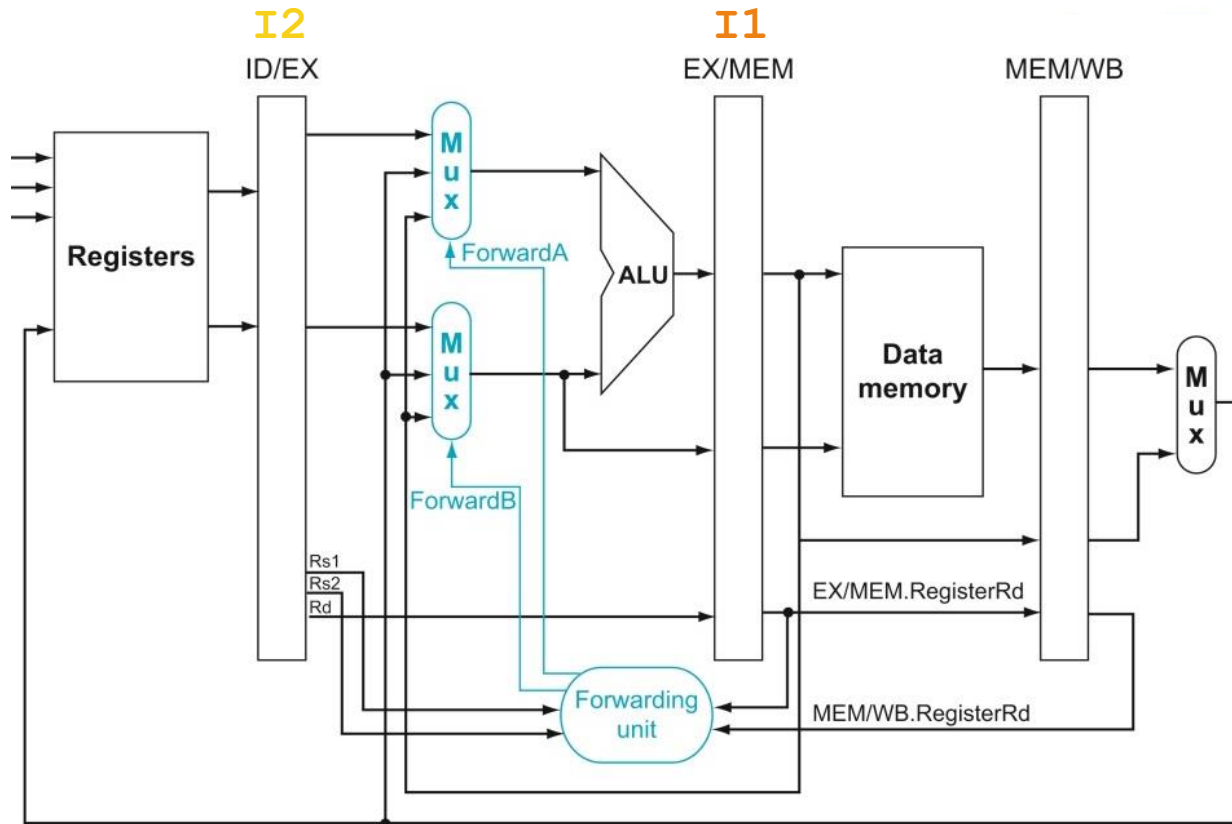
x1	5
x2	10
x3	3
x4	4
x5	5

I1: add **x3**, x1, x2

I2: sub x4, **x3**, x2

I3: mul x5, x2, **x3**

Slutten av
klokkesykel 2



Eksempel: Videresending

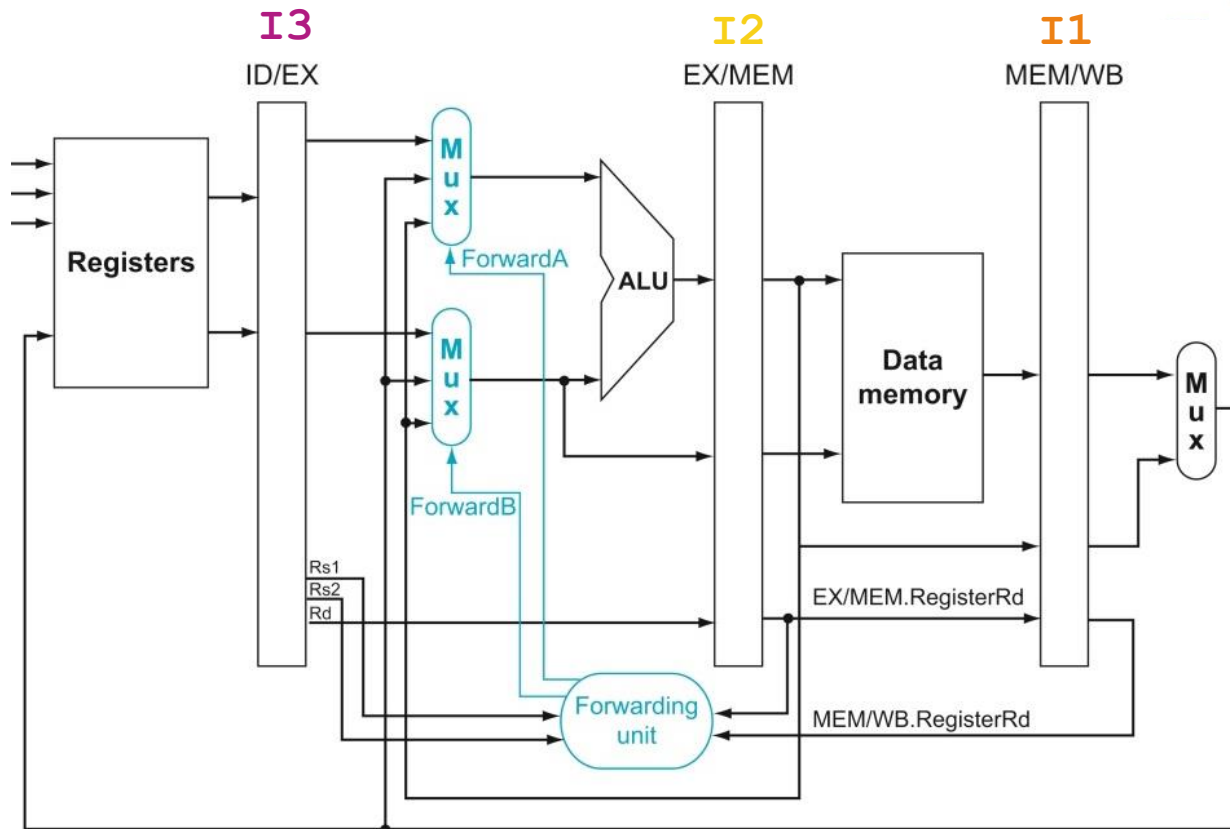
x1	5
x2	10
x3	3
x4	4
x5	5

I1: add **x3**, x1, x2

I2: sub x4, **x3**, x2

I3: mul x5, x2, **x3**

Slutten av
klokkesykel 3



Eksempel: Videresending

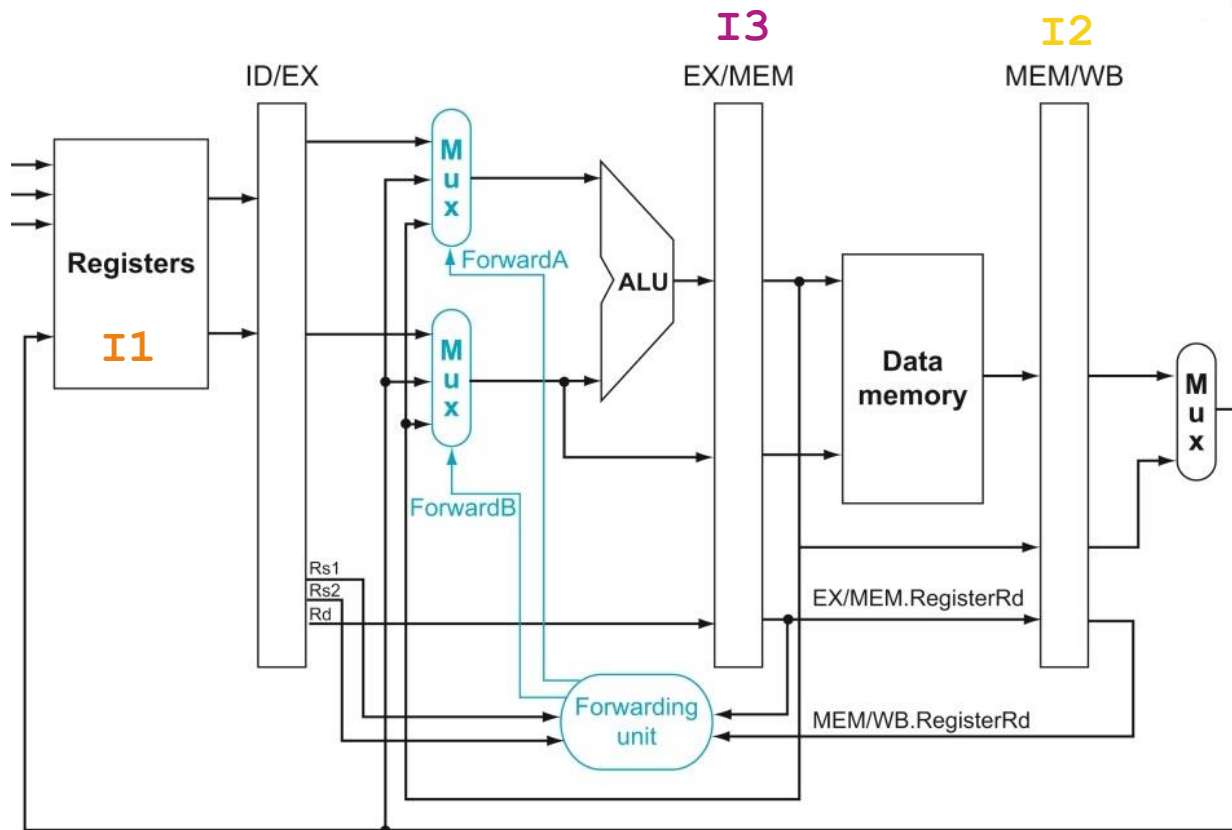
x1	5
x2	10
x3	15
x4	4
x5	5

I1: add **x3**, x1, x2

I2: sub x4, **x3**, x2

I3: mul x5, x2, **x3**

Slutten av
klokkesykel 4



Eksempel: Videresending

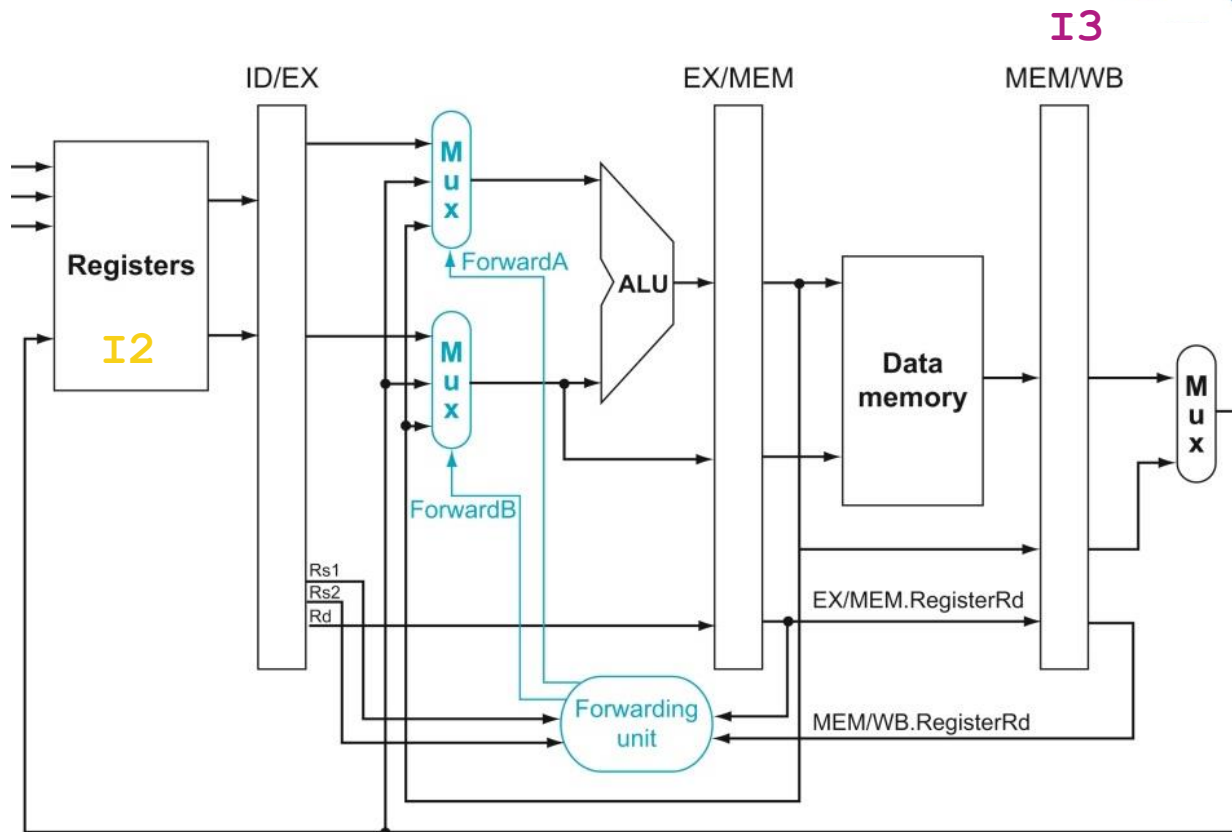
x1	5
x2	10
x3	15
x4	5
x5	5

I1: add **x3**, x1, x2

I2: sub x4, **x3**, x2

I3: mul x5, x2, **x3**

Slutten av
klokkesykel 5



Eksempel: Videresending

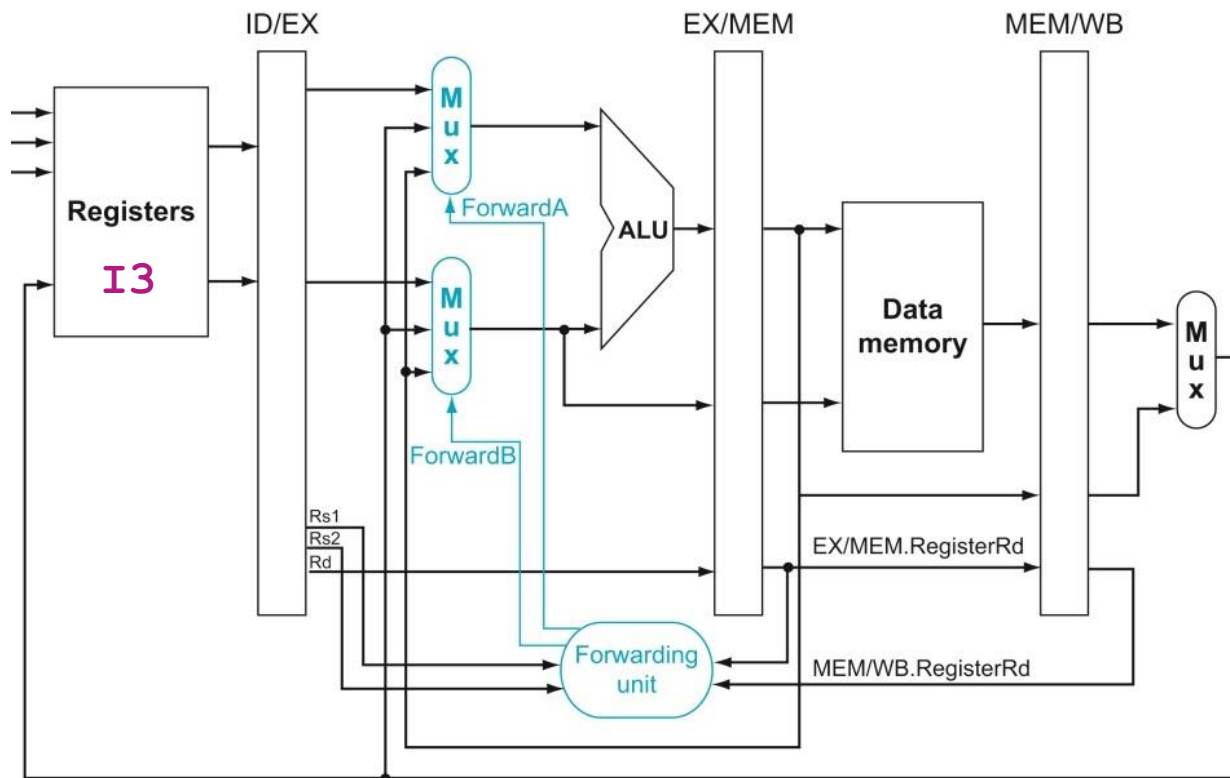
x1	5
x2	10
x3	15
x4	5
x5	150

I1: add **x3**, x1, x2

I2: sub x4, **x3**, x2

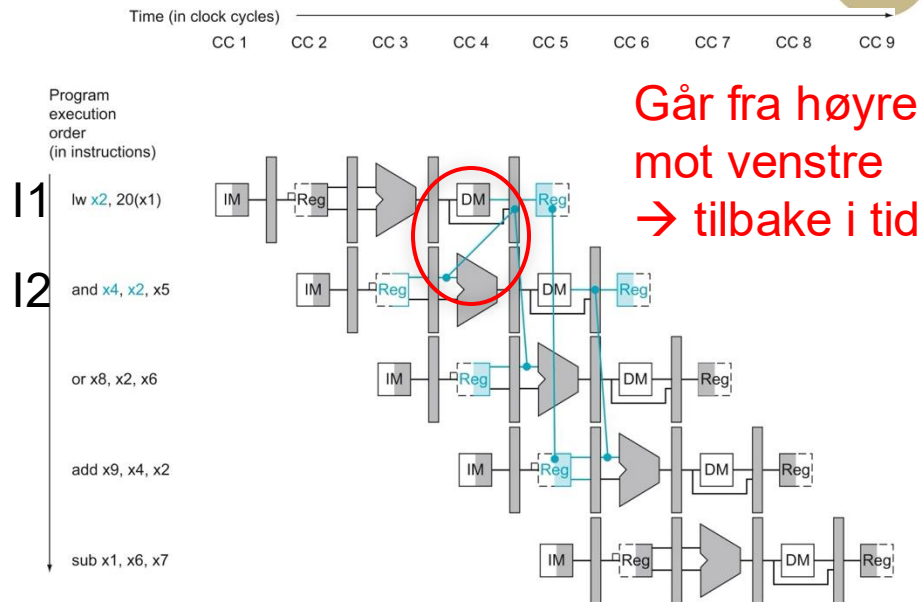
I3: mul x5, x2, **x3**

Slutten av
klokkesykel 6



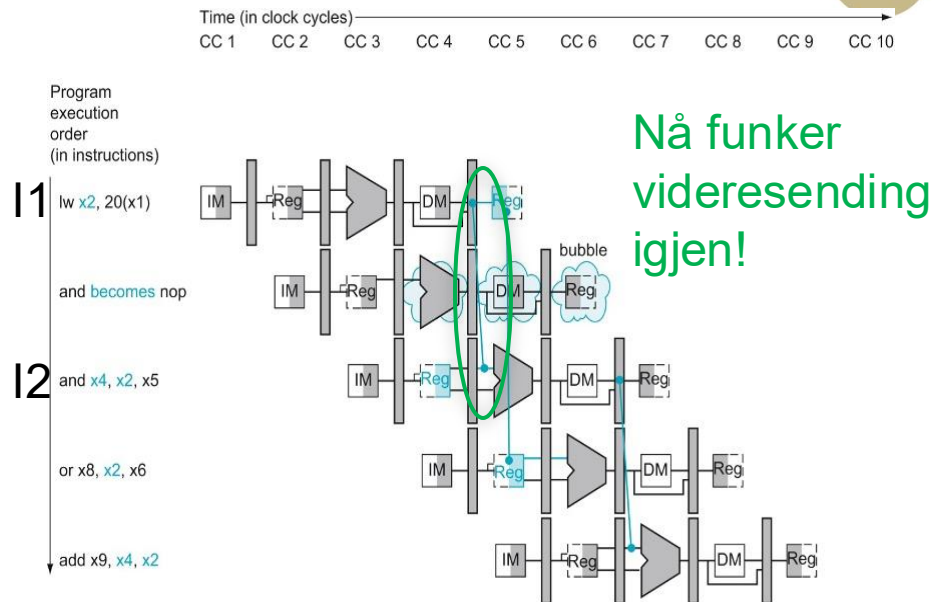
Les-bruk farer («Load-use hazards»)

- Videresending fungerer bare når verdien har blitt produsert når vi trenger den
- Dette eksemplet: I2 trenger verdien i x2 før I1 har hentet den fra minnet
 - Dette er en les-bruk fare
 - Videresending kan ikke brukes fordi vi kan ikke sende verdier tilbake i tid
- Løsning: Vi må holde igjen I2
 - ... med enda mer maskinvare



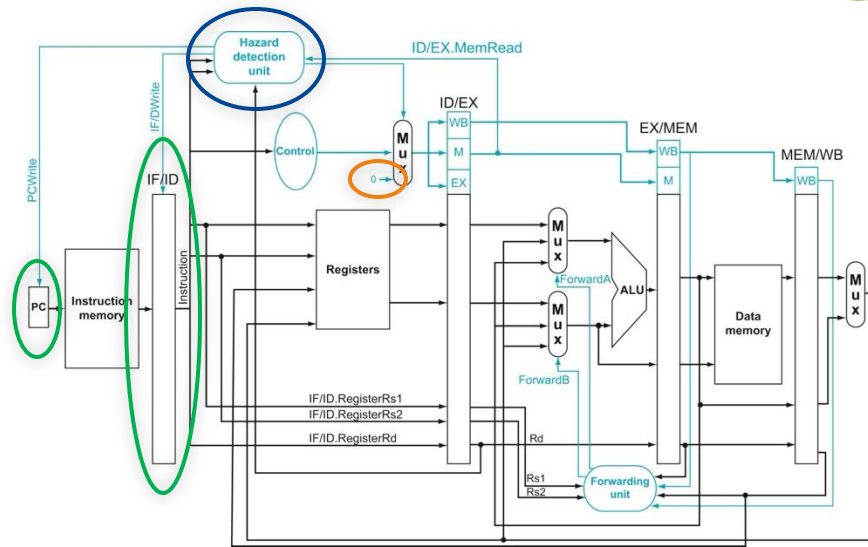
Les-bruk farer («Load-use hazards»)

- Videresending fungerer bare når verdien har blitt produsert når vi trenger den
- Dette eksemplet: I2 trenger verdien i x2 før I1 har hentet den fra minnet
 - Dette er en les-bruk fare
 - Videresending kan ikke brukes fordi vi kan ikke sende verdier tilbake i tid
- Løsning: Vi må holde igjen I2
 - ... med enda mer maskinvare



Faredeteksjon («Hazard detection»)

- Faredeteksjonsenheten:
 - Skal vi lese fra minnet?
 - Skal vi skrive til et register som neste instruksjon leser?
- Hvis svaret på begge disse spørsmålene er ja:
 - Lag et hull i samlebandet ved sette (nødvendige) kontrollsignaler til 0
 - Hold samme verdi i PC og IF/ID



Eksempel: Stans

Registerfil

x1	1024
x2	5
x3	1
x4	-1

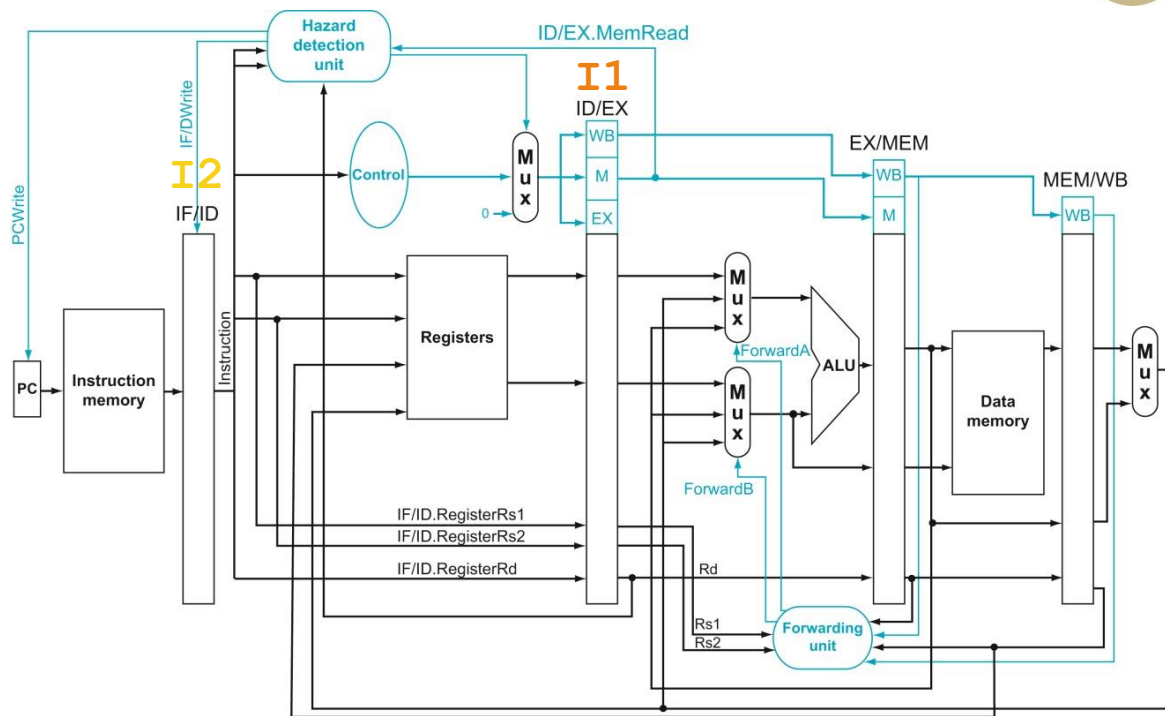
Minnet

1032	-
1028	-
1024	10
1020	-

I1: lw **x3**, 0(x1)

I2: add x4, **x3**, x2

Slutten av
klokkesykel 1



Eksempel: Stans

Registerfil

x1	1024
x2	5
x3	1
x4	-1

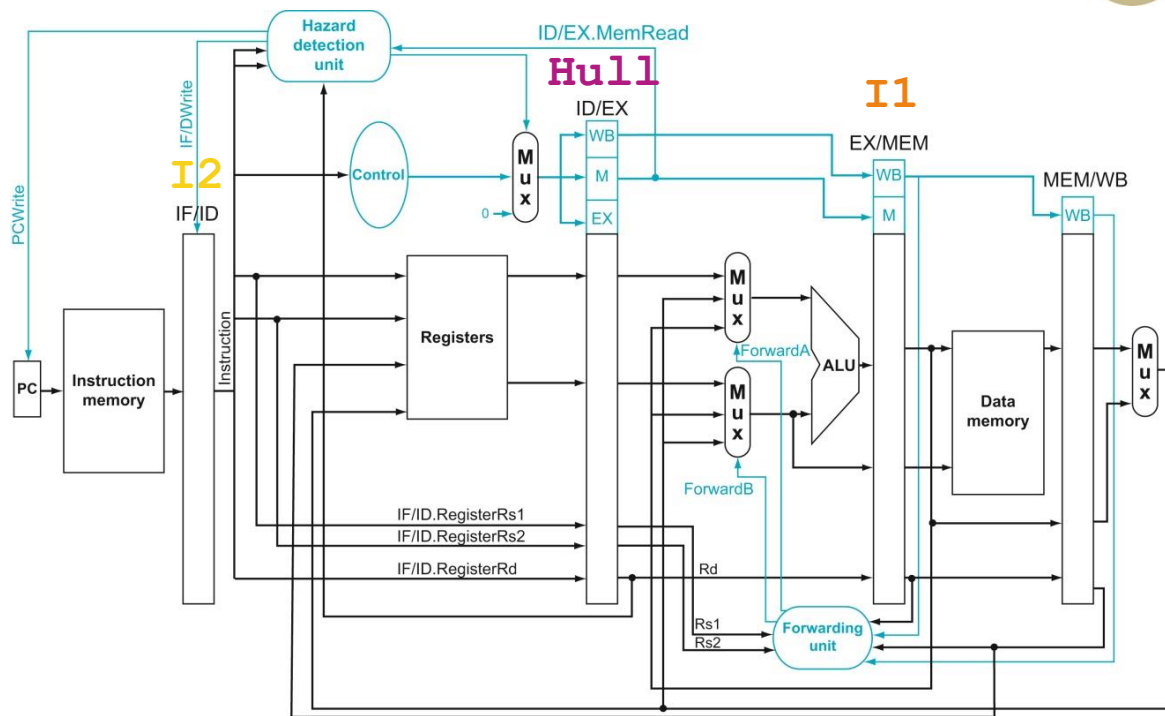
Minnet

1032	-
1028	-
1024	10
1020	-

I1: lw **x3**, 0(x1)

I2: add x4, **x3**, x2

Slutten av
klokkesykel 2



Eksempel: Stans

Registerfil

x1	1024
x2	5
x3	1
x4	-1

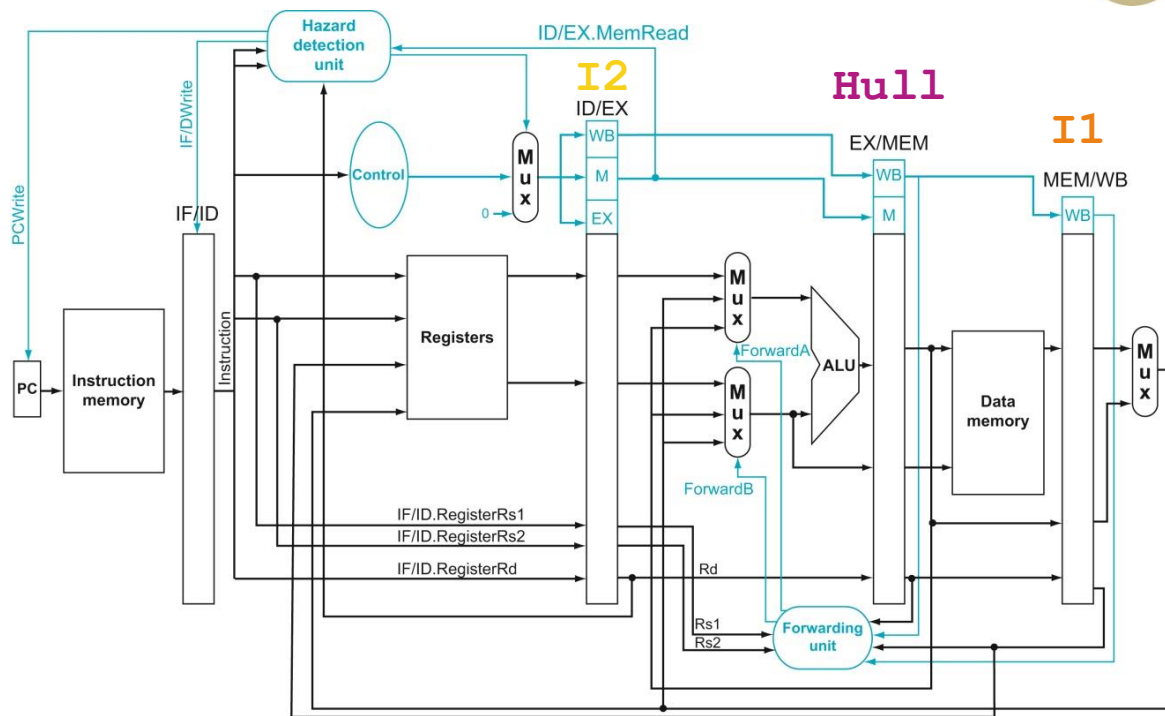
Minnet

1032	-
1028	-
1024	10
1020	-

I1: lw **x3**, 0(x1)

I2: add x4, **x3**, x2

Slutten av
klokkesykel 3



Eksempel: Stans

Registerfil

x1	1024
x2	5
x3	10
x4	-1

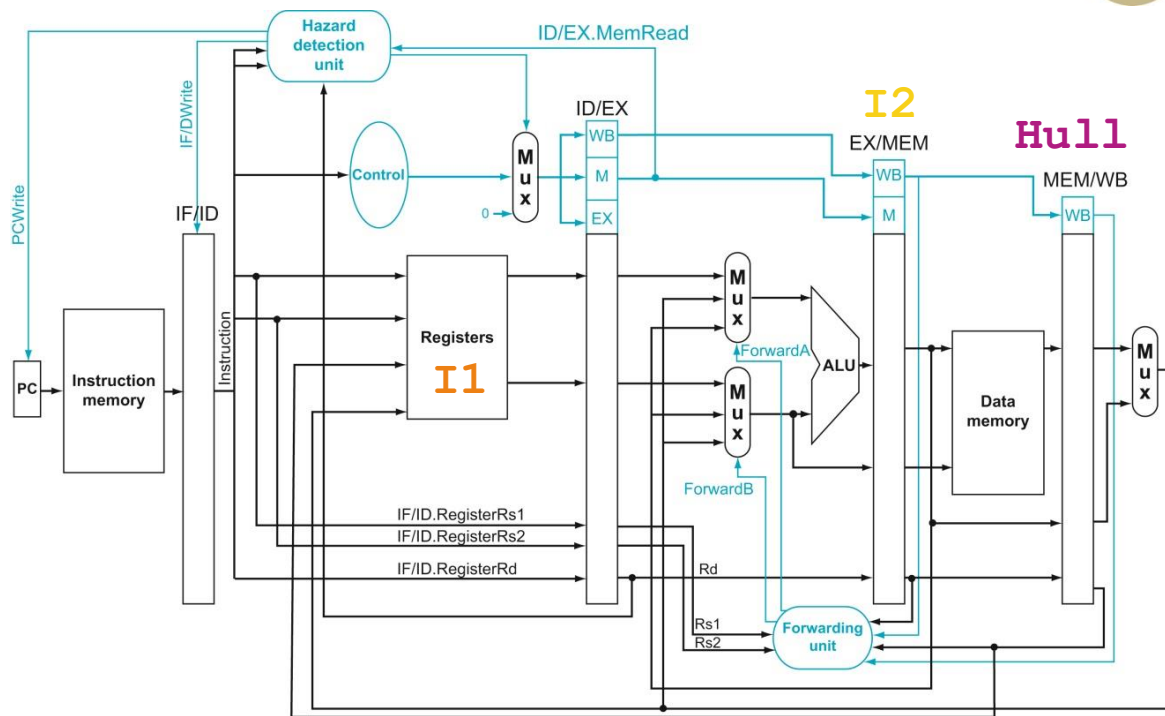
Minnet

1032	-
1028	-
1024	10
1020	-

I1: lw **x3**, 0(x1)

I2: add x4, **x3**, x2

Slutten av
klokkesykel 4



Eksempel: Stans

Registerfil

x1	1024
x2	5
x3	10
x4	-1

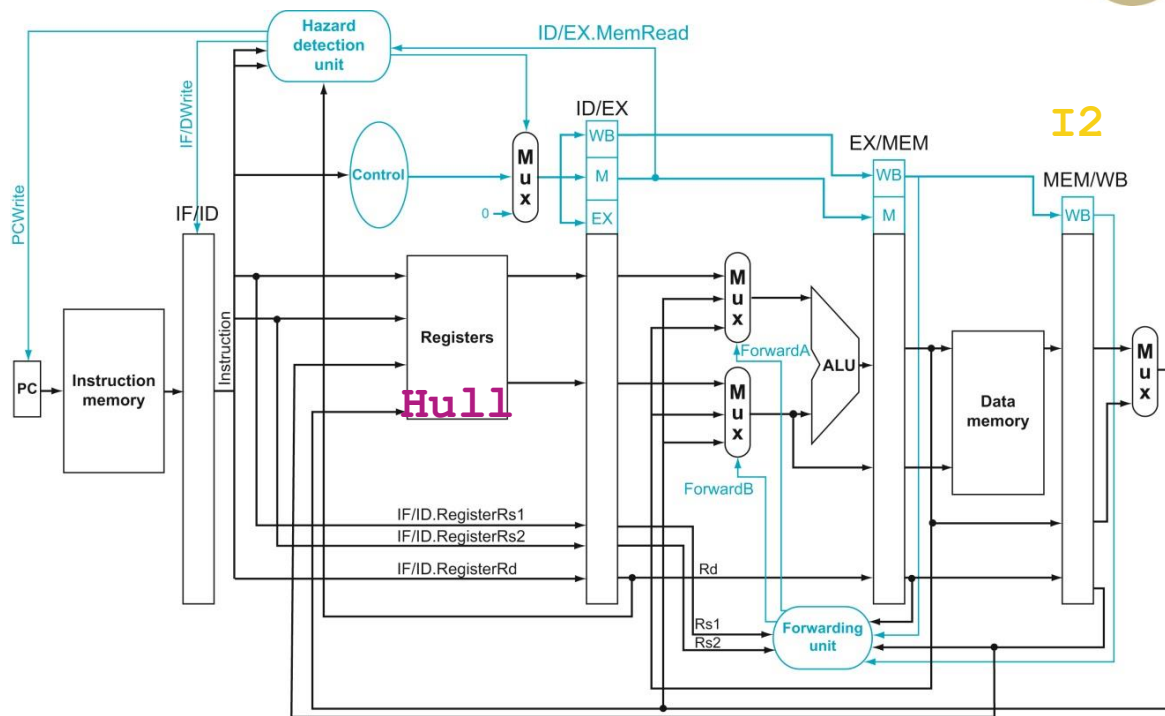
Minnet

1032	-
1028	-
1024	10
1020	-

I1: lw **x3**, 0(x1)

I2: add x4, **x3**, x2

Slutten av
klokkesykel 5

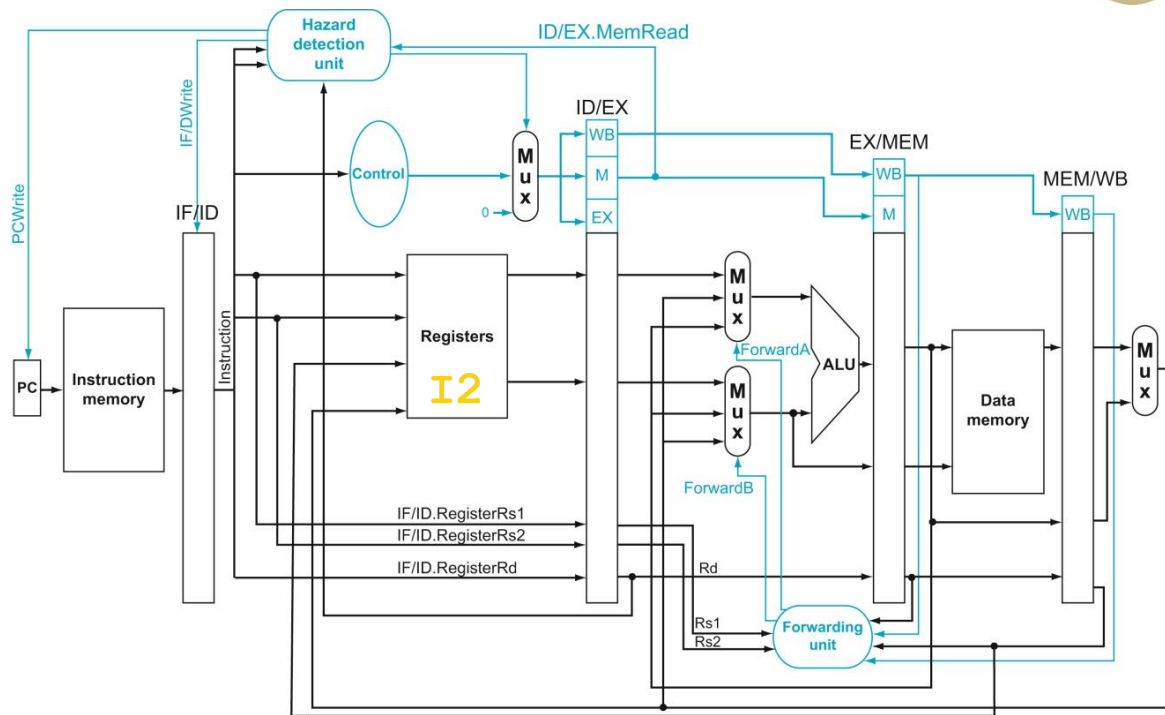


Registerfil		Minnet	
x1	1024	1032	-
x2	5	1028	-
x3	10	1024	10
x4	15	1020	-

I1: $1_w \times 3, 0 (\times 1)$

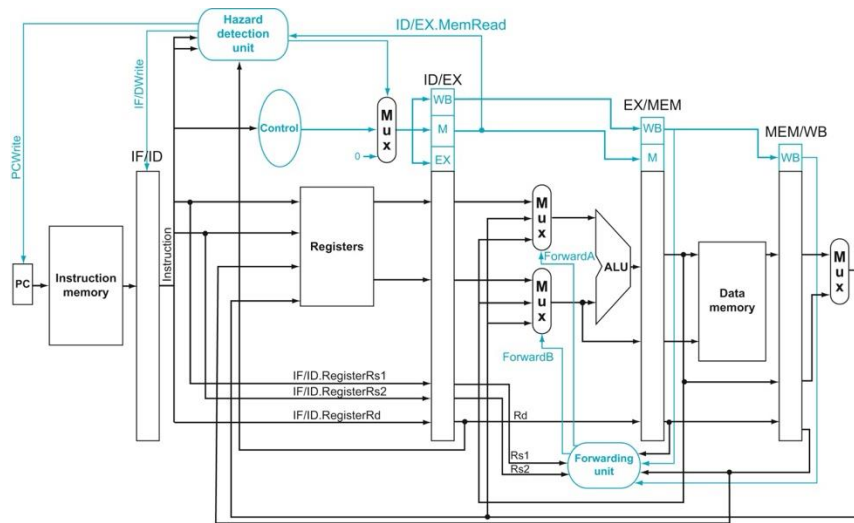
```
I2: add x4, x3, x2
```

Slutten av klokkesykel 6



Datafarer: Oppsummering

- Vi kan nå håndtere alle datafarer
 - Vi bruker videresending så mye som mulig for å unngå ytelsestap
 - Vi stanser samlebåndet når vi må
- Kompilatoren trenger derfor ikke å bry seg om datafarer for korrekthet
 - ... men for ytelse lønner det seg å ikke generere kode med les-bruk farer





Tema 5.1 Samlebåndspresessor med 5 steg (Kapittel 4.9)

KONTROLLFARER

Læringsutbytte T5.1

- Studenten skal kunne beskrive fordeler og ulemper ved samlebåndsprosessoren sammenliknet med flersykelprosessoren og enkeltsykelprosessoren.
- Studenten skal kunne forklare hvordan oppstartskostnad og balanse mellom steg i samlebåndet påvirker ytelse.
- Studenten skal kunne beskrive forskjellen mellom avhengigheter og farer samt gjengi de tre hovedgruppene av farer (strukturfarer, datafarer, og kontrollfarer).
- Studenten skal kjenne til de fire overordnede strategiene for å håndtere farer (unngåelse, videresending, stans, og prediksjon)
- Studenten skal kunne forklare den grunnleggende mikroarkitekturen til 5-stegs samlebåndsprosessoren (kunne gjengi og forklare figur 4.43).
- Studenten skal kunne forklare hvordan kontroll implementeres i 5-stegs samlebåndsprosessoren.
- Studenten skal kunne forklare hvordan videresending og stans kan brukes til å håndtere datafarer samt kunne analysere hvordan strategiene påvirker prosessorens ytelse.
- Studenten skal kunne forklare hvordan **kontrollfarer** kan **håndteres** med **stans** og **prediksjon** samt analysere hvordan strategiene påvirker prosessorens ytelse.

Kontrollfarer

Menti!

- Kontrollfarer oppstår ved betingede forgreiningsinstruksjoner
 - ... fordi vi vet ikke vi skal utføre neste instruksjon eller den instruksjonen vi hopper til før etter EX-steget
- Å alltid stanse («Stalling») gir en uakseptabel ytelsesreduksjon
 - ... fordi vi mister 2 klokkesykler på hver forgreiningsinstruksjon
 - ... og forgreiningsinstruksjoner er vanlige

Stans på alle betingede forgreiningsinstruksjoner

	1	2	3	4	5	6	7	8
Beq	IF	ID	EX	MEM	WB			
-		IF	ID	EX	MEM	WB		
-			IF	ID	EX	MEM	WB	
Add				IF	ID	EX	MEM	WB

Perfekt forgreiningsprediksjon (ikke realiserbart)

	1	2	3	4	5	6	7	8
Beq	IF	ID	EX	MEM	WB			
Add		IF	ID	EX	MEM	WB		

Eksempel: Forgreining og ytelse

Ideell ytelse for samlebåndsprosessoren vår $\rightarrow \text{CPI} = 1$

Anta at alle andre instruksjoner enn forgreiningsinstruksjoner har $\text{CPI} = 1$



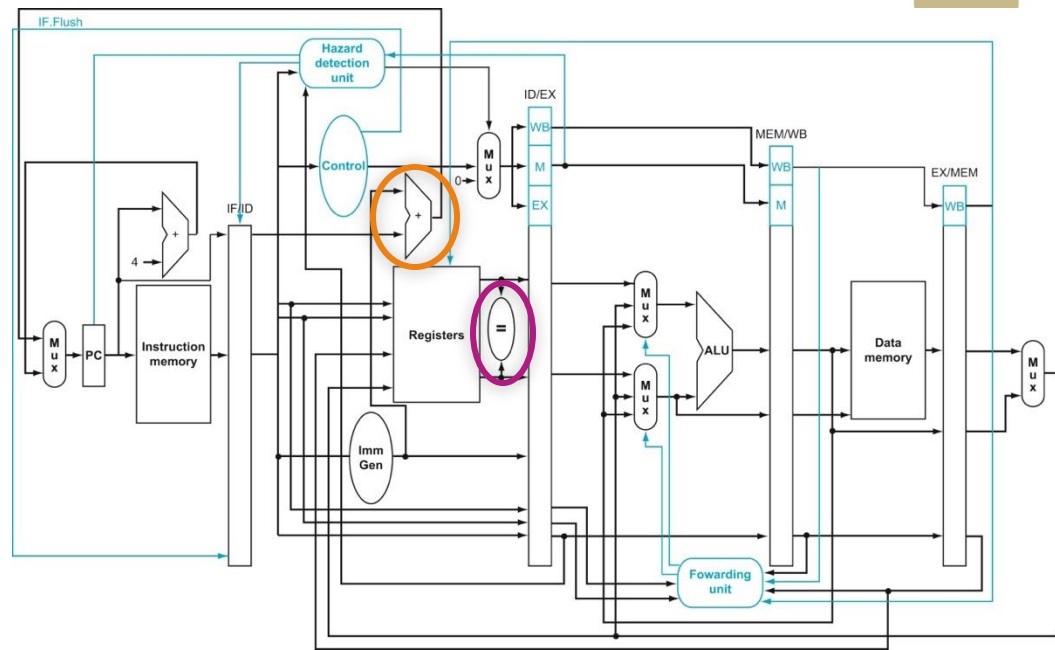
Forgreningsprediksjon («Branch prediction»)



- Det enkleste er å ta sjansen på at forgreiningsinstruksjonen ikke gir hopp («Predict not taken»)
 - ... for da skal vi bare hente neste instruksjon slik vi vanligvis gjør
 - ... og da sparer vi i hvert fall tid i ett av tilfellene
- Hvis vi tar feil, må vi fjerne instruksjonene som ikke skal utføres fra samlebåndet («flush the pipeline»)
 - Akkurat som ved les-skriv farer gjør vi dette ved å sette kontrollsignaler til 0 i samlebåndsregistrene
 - Men: Vi må gjøre dette for IF, ID, og EX

Detektere forgreining tidlig

- Jo tidligere i samlebåndet vi vet om vi skal hoppe eller ikke, jo mindre taper vi når vi gjetter feil
 - ... og vi trenger ikke en hel ALU til å sjekke om registerverdiene er like
- Optimalisering: Utføre `beq` i ID
 - (i) Regne ut måladressen («branch target»)
 - (ii) Sjekke om registerverdiene er like
- .. men vi trenger også mer
 - Videresending (f. eks. EX/MEM → ID)
 - Faredeksjon (EX → `beq`, MEM → `beq`)



Eksempel: Fjerne instruksjoner

Registerfil

x1	15
x2	15

I1 (0) : beq x1, x2, 16

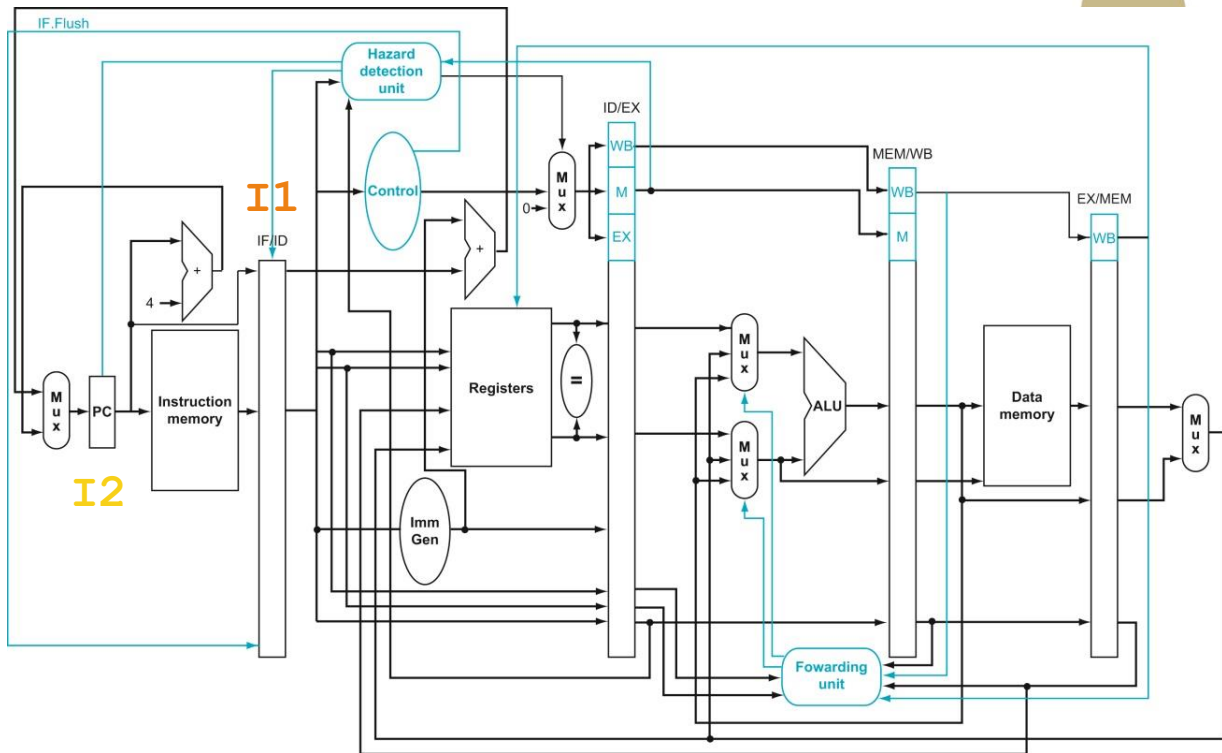
I2 (4) : add x3, x1, x2

I3 (8) : ...

I4 (12) : ...

I5 (16) : ...

Slutten av
klokkesykel 1



Eksempel: Fjerne instruksjoner

Registerfil

x1	15
x2	15

I1 (0) : beq x1, x2, 16

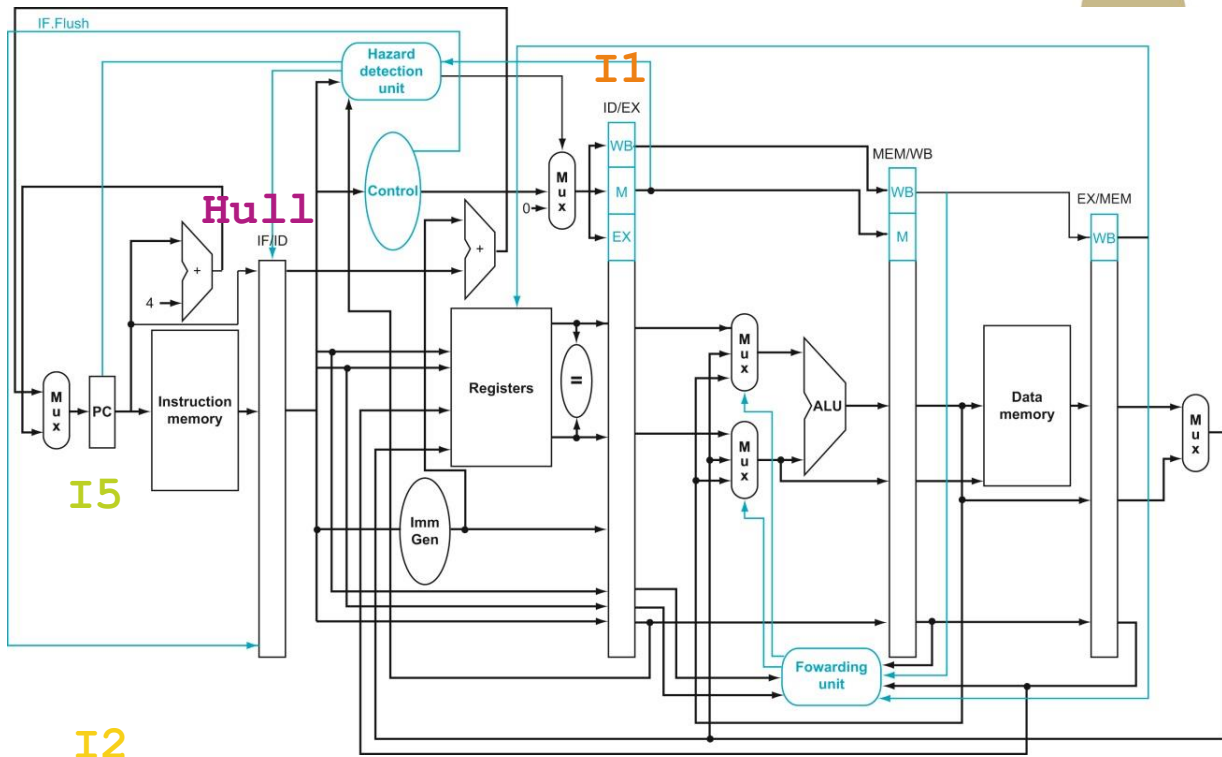
I2 (4) : add x3, x1, x2

I3 (8) : ...

I4 (12) : ...

I5 (16) : ...

Slutten av
klokkesykel 2



Eksempel: Fjerne instruksjoner

Registerfil

x1	15
x2	15

I1 (0) : beq x1, x2, 16

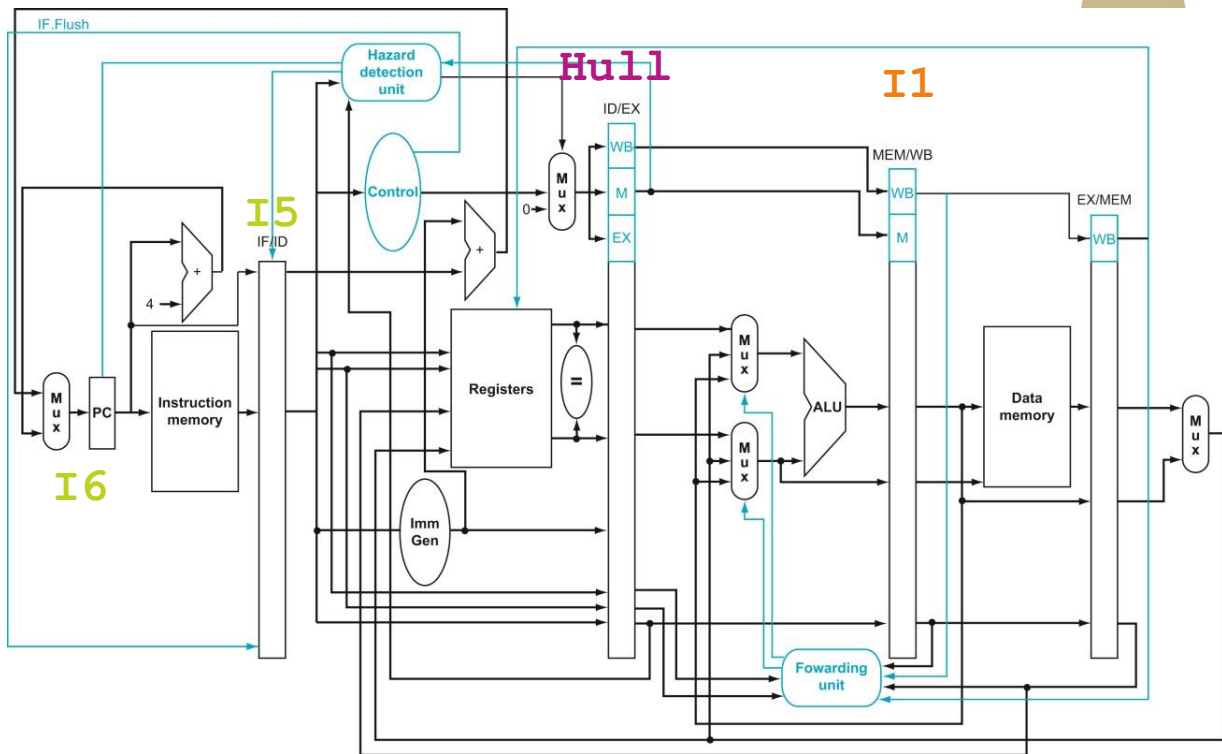
I2 (4) : add x3, x1, x2

I3 (8) : ...

I4 (12) : ...

I5 (16) : ...

Slutten av
klokkesykel 3



Eksempel: Fjerne instruksjoner

Registerfil

x1	15
x2	15

I1 (0) : beq x1, x2, 16

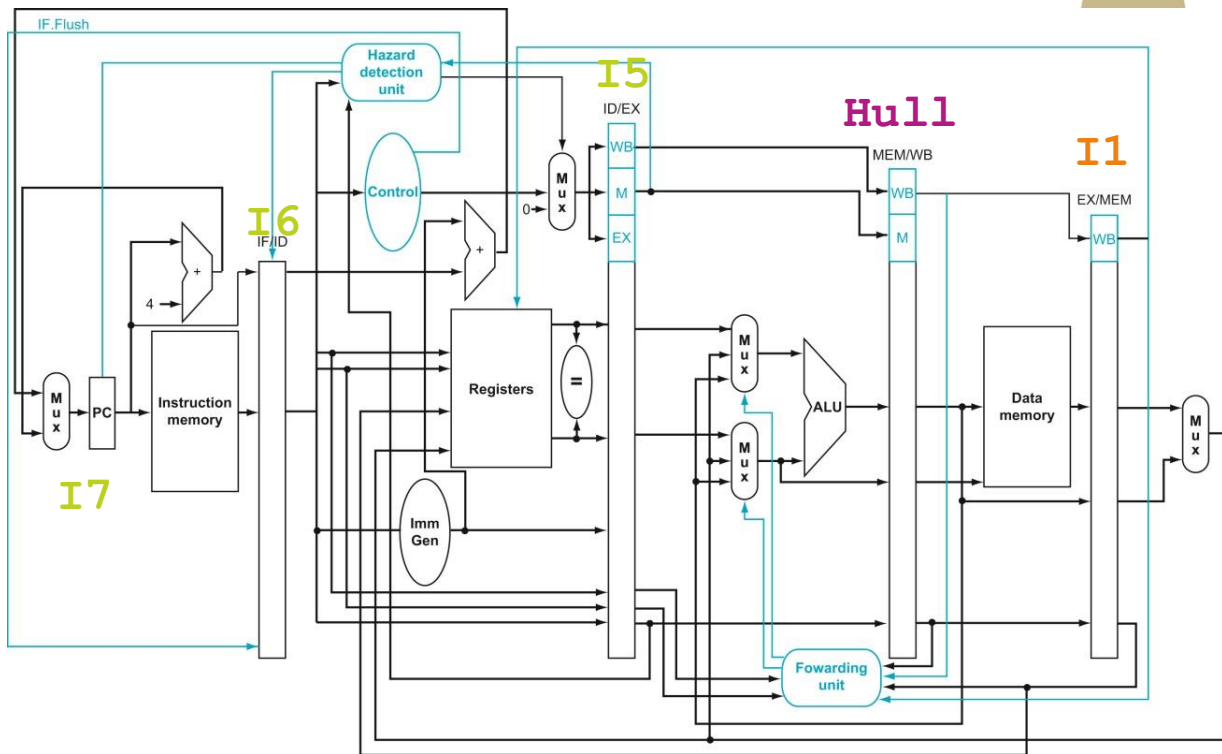
I2 (4) : add x3, x1, x2

I3 (8) : ...

I4 (12) : ...

I5 (16) : ...

Slutten av
klokkesykel 4



Registerfil

x1	15
x2	15

```
I1 (0) : beq x1, x2, 16
```

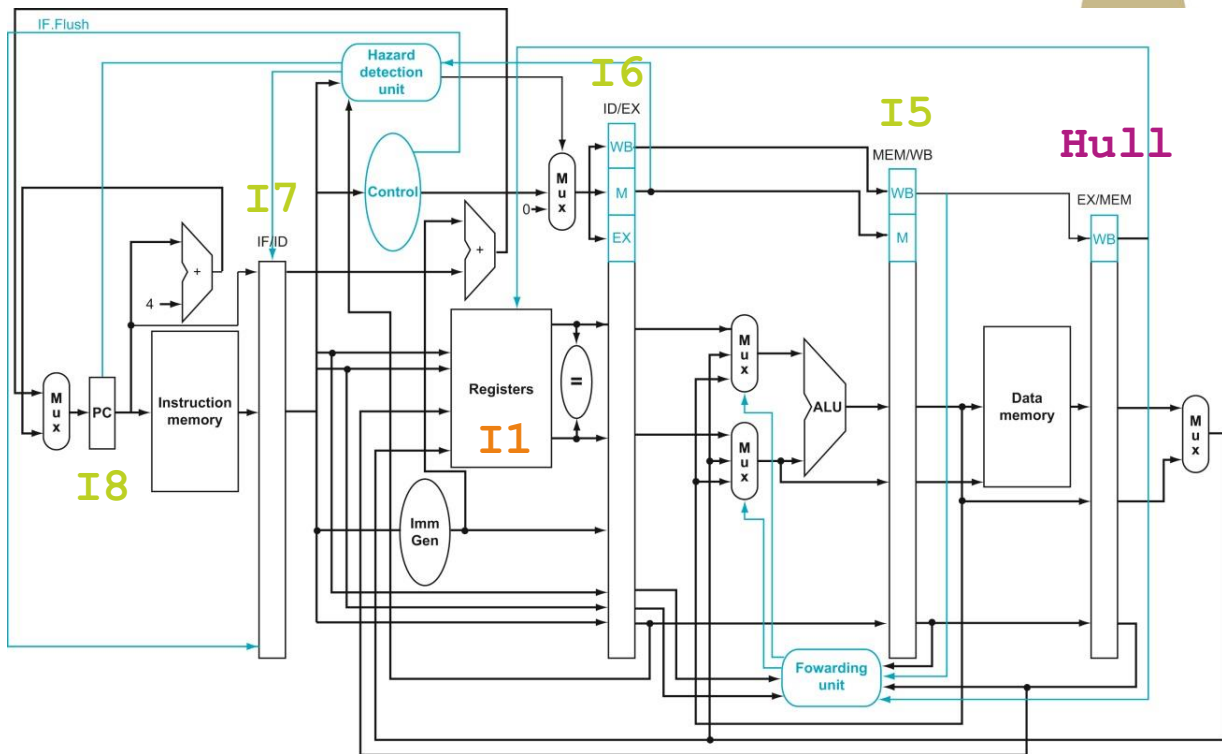
```
I2 (4) : add x3, x1, x2
```

I3 (8) : ...

$$I_4(12) : \dots$$

I5 (16) : ...

Slutten av klokkesykel 5



Registerfil

x1	15
x2	15

```
I1 (0) : beq x1, x2, 16
```

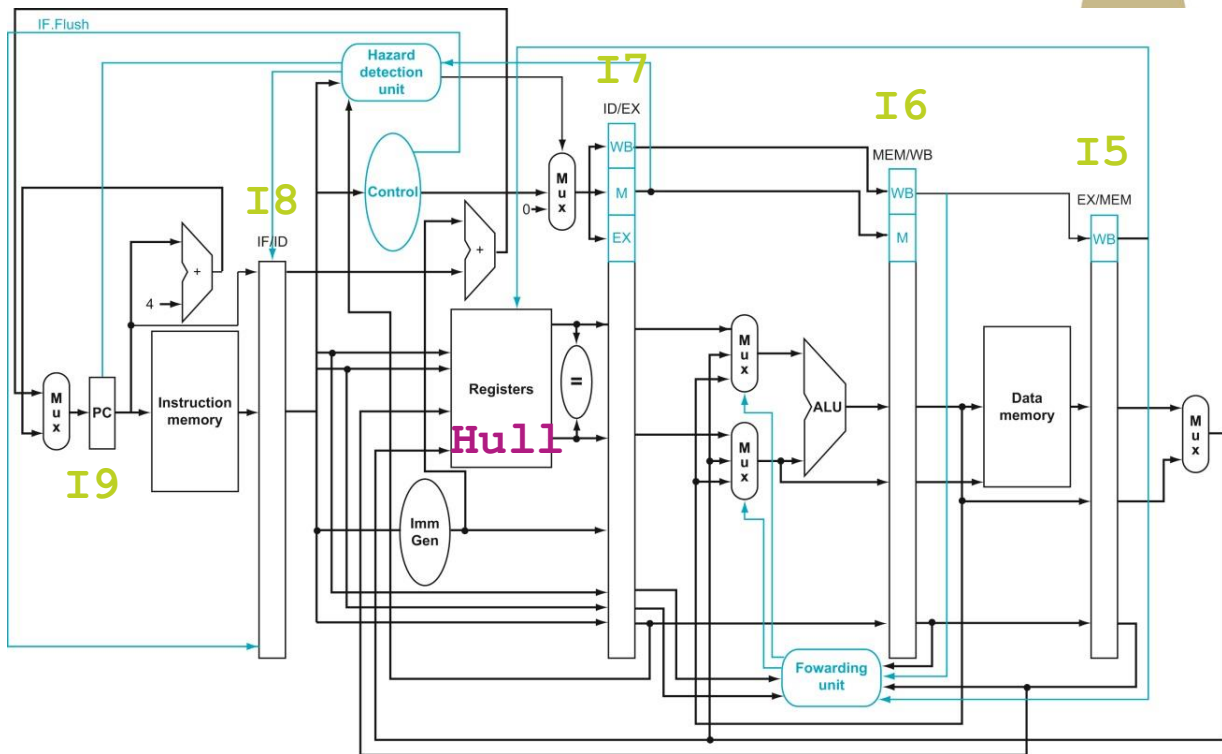
```
I2 (4) : add x3, x1, x2
```

I3 (8) : ...

I4 (12) : ...

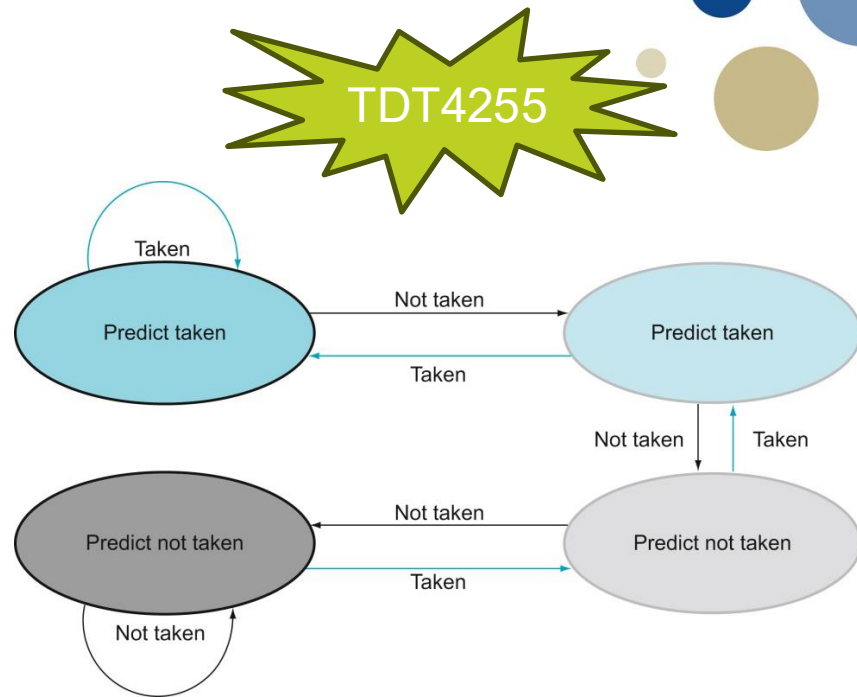
I5 (16) : . . .

Slutten av klokkesykel 6



Mer avansert forgreningsprediksjon

- Observasjon: Det er ikke tilfeldig om en forgreiningsinstruksjon hopper eller ikke
- Dynamisk forgreiningsprediksjon bruker maskinvare til å holde styr på:
 - Hvor hoppet forgreiningsinstruksjonen sist? («Branch Target Buffer (BTB)»)
 - Hva gjorde denne forgreiningsinstruksjonen de siste n gangene vi har sett den? (lokal historie)
 - ... og/eller hva har de siste m forgreningsinstruksjonene gjort? (global historie)



2-bit «bimodal» forgreiningsprediktor

Forgreningsprediktorene i høytytelsesmaskiner gjetter nesten ikke feil (>95% rett)



Tema 5.2: Unntak og avbrudd (Kapittel 4.10)

UNNTAK OG AVBRUDD

Læringsutbytte T5.2

- Studenten skal kunne forklare begrepene **unntak** og **avbrudd**, inkludert **presise unntak**.
- Studenten skal kunne beskrive hvordan **presise unntak** kan implementeres i 5-stegssamlebåndet.
- Studenten skal kjenne til prinsippene bak hvordan presise unntak implementeres i prosessorer som utfører instruksjoner ut-av-rekkefølge.

Unntak og avbrudd

- Unntak («Exception»)
 - En ikke-forventet endring i kontrollflyt
 - Eksempel: En ikke-implementert instruksjon
- Avbrudd («Interrupt»)
 - Et unntak der kilden er utenfor prosessoren (for eksempel I/O)
- Disse ordene brukes ikke konsistent, men jeg (og boka) syns det gir mening å skille mellom interne og eksterne kilder

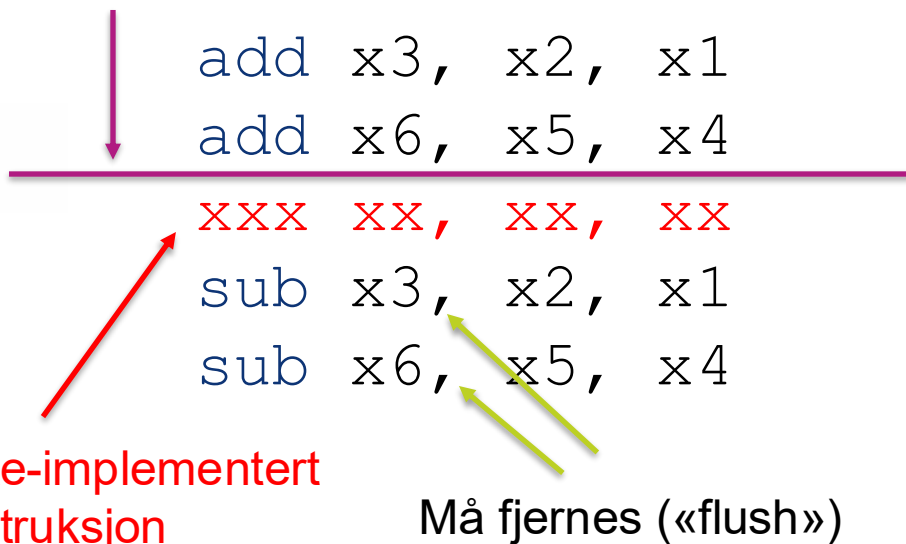
Hvordan håndtere unntak?

- Når et unntak oppstår i en RISC-V maskin, må vi:
 1. Lagre adressen til instruksjonen som utføres i et forhåndsdefinert register (SEPC)
 2. Lagre grunnen til unntaket i et forhåndsdefinert register (SCAUSE)
 3. Overføre kontroll til operativsystemet på en forhåndsdefinert adresse
 4. Operativsystemet håndterer unntaket
- Høres det kjent ut?
 - Det er ikke så rart fordi det er veldig likt et vanlig funksjonskall
 - Forskjellen er at applikasjonen ikke er involvert, dvs at prosessoren og operativsystemet samarbeider om å håndtere unntaket
- Alternativ: Vektoriserte unntak
 - Sløyfer skritt 2 og hopper heller til ulike adresser for ulike unntak

Unntak og samleband

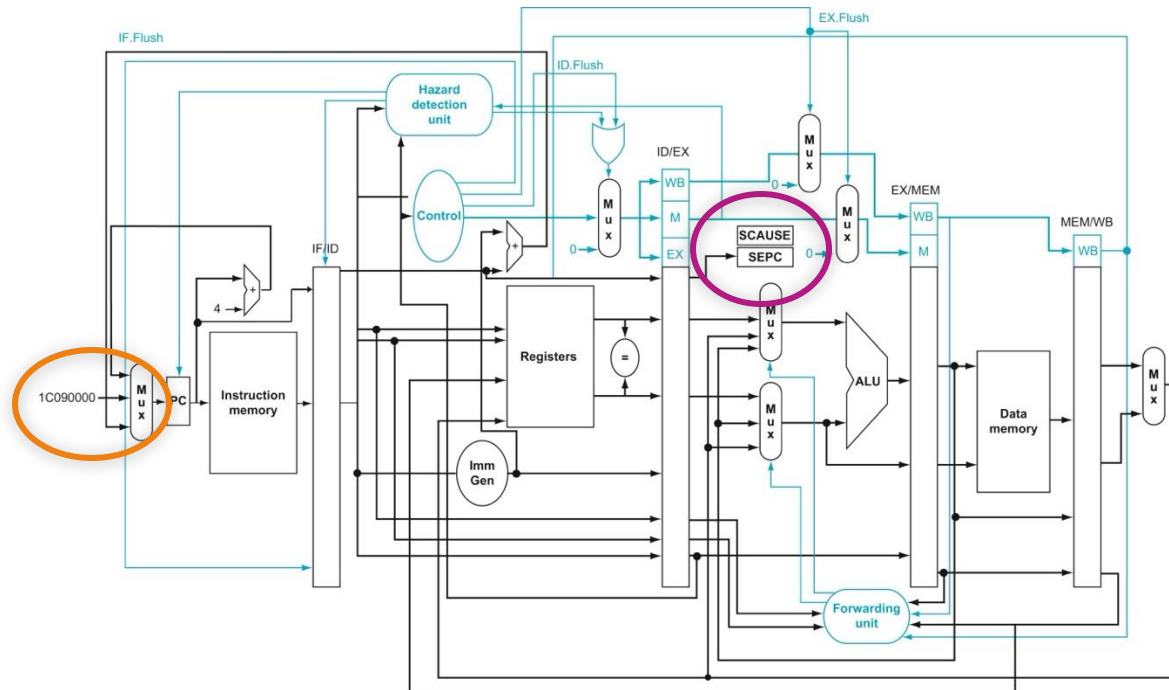
- For at operativsystemet skal gjøre jobben sin må tilstanden til maskinen være som om:
 - Alle instruksjoner før er ferdige
 - Ingen instruksjoner etter har kjørt
- Dette kalles **presise unntak** («**precise exceptions**») og gjør det mulig å starte programmet på nytt fra der unntaket skjedde
 - ... og dette utnytter vi for eksempel til å implementere virtuelt minne

Alle instruksjoner før har kjørt



Implementere støtte for unntak

1. Lagre adressen til instruksjonen som utføres i et forhåndsdefinert register (SEPC)
2. Lagre grunnen til unntaket i et forhåndsdefinert register (SCAUSE)
3. Overføre kontroll til operativsystemet på en forhåndsdefinert adresse
4. Operativsystemet håndterer avviket





Tema 5.3: Prosessorer med høyere ytelse (Kapittel 4.11)

SUPERSKALRITET

Læringsutbytte T5.3

- Studenten skal kunne beskrive hvordan en prosessor utnytter **parallellitet i tid** og **parallellitet i rom**.
- Studenten skal kunne gjenkjenne og beskrive **RAW, WAW og WAR farer** og avhengighetene som skaper dem.
- Studenten skal forstå prinsippene bak hvordan **«register renaming»** fjerner WAW og WAR farer.
- Studenten skal kjenne til fordeler og ulemper med **«static multi-issue» prosessorer**.
- Studenten skal kjenne til prinsippene bak hvordan prosessorer som utfører instruksjoner ut av rekkefølge («out of order») implementeres («dynamic multi-issue»).
- Studenten skal kunne forklare begrepet «speculation».

5-stegs samlebandet vårt har noen utfordringer



- Vi har antatt at alle R-type instruksjoner tar like lang tid
 - ... men heltallsaddisjon krever mye enklere maskinvare enn en flyttallsaddisjon og kan dermed ta kortere tid
 - ... og klokkefrekvensen er avhengig av den tregeeste operasjonen
- Vi kan maksimalt utføre en instruksjon per klokkesykel (maksimal IPC = 1)
- Vi utfører instruksjonene i den rekkefølgen de er i programmet
 - ... og da går vi glipp av mye parallellitet

5-stegs samlebåndet vårt har noen utfordringer



- Vi har antatt at alle R-type instruksjoner tar like lang tid
 - ... men heltallsaddisjon krever mye enklere maskinvare enn en flyttallsaddisjon og kan dermed ta kortere tid
 - ... og klokkefrekvensen er avhengig av den tregeeste operasjonen
- Vi kan maksimalt utføre en instruksjon per klokkesykel (maksimal IPC = 1)
- Vi utfører instruksjonene i den rekkefølgen de er i programmet
 - ... og da går vi glipp av mye parallellitet

Høyere ytelse → Mer parallellitet



- Samlebånd utnytter parallellitet i tid
 - Hvert steg jobber med ulike instruksjoner
 - Vi kan oppnå høyere ytelse gjennom å gjøre samlebåndet dypere
 - Krever lite ekstra maskinvare
- Superskalar utføring utnytter parallellitet i rom
 - Et skritt jobber med flere instruksjoner samtidig
 - Krever mye ekstra maskinvare (for eksempel en ekstra ALU)
- Superskalare samlebånd utnytter parallellitet i tid og rom

Et superskalart samleband



Flere dataavhengigheter og datafarer

- Vi har så langt sett på sanne dataavhengigheter
 - En instruksjon leser det en annen skriver
 - Gir «Read-After-Write (RAW)» farer
- Når vi utfører flere instruksjoner samtidig, må vi også håndtere navneavhengigheter:
 - Ut-avhengigheter som gir «Write-After-Write (WAW)» farer
 - Anti-avhengigheter som gir «Write-After-Read (WAR)» farer

RAW

```
add  x3, x2, x1
sub  x7, x3, x6
```

WAW

```
add  x3, x2, x1
sub  x3, x5, x6
```

WAR

```
add  x3, x2, x1
sub  x2, x3, x6
```

WAW eksempel

WAW

mul	x3,	x2,	x1
	↓		
sub	x3,	x5,	x6

Multiplikasjon tar 3 sykler
Subtraksjon tar 1 sykel

«Register renaming»

- Navneavhengigheter oppstår fordi vi har et begrenset antall registre
 - ... men vi har allerede gjort så mange registre som vi kan tilgjengelig for programmet da vi laget instruksjonssettet
- Løsning: «Register Renaming»
 - Vi legger til (vesentlig) flere fysiske registre i prosessoren enn de programmet kan se (arkitekturregistre)
 - Vi tilordner fysiske registre til instruksjoner slik at vi
 - Respekterer sanne dataavhengigheter (RAW-farer)
 - Fjerner navneavhengigheter (WAW og WAR-farer)
- Nøkkelobservasjon: Hver instruksjon skriver til maksimalt ett register



add x3, x2, x1

add x3, x5, x4

add x5, x2, x4

sub x7, x3, x5

«Register renaming»

Hver instruksjon får et nytt fysisk målregister

WAW-faren er borte!

add x3, x2, x1

add x3, x5, x4

add x5, x2, x4

sub x7, x3, x5

add p6, p2, p1

add p7, p4, p3

add p8, p2, p3

sub p9, p7, p8

WAR-faren er borte!

Kun RAW-farer er igjen

TDT4255

Tilordningstabell

x1 → p1

x2 → p2

x4 → p3

x5 → p4

x3 → p6 (Gammel)

x3 → p7

x5 → p8

x7 → p9

Kompilatoren kan også finne uavhengige instruksjoner



- Ide: Kompilatoren kan generere programmet på en måte som forteller prosessoren om instruksjonene kan utføres samtidig
 - Instruksjonspakke: Alle instruksjoner i samme pakke kan utføres samtidig
 - «Very Long Instruction Word (VLIW)» og «Static multi-issue»
- Rene VLIW prosessorer er ikke vanlige
 - 👍 Maskinvaren blir enklere (se figur 4.70 i boka)
 - 👎 Kompilatoren må jobbe hardt for å utnytte instruksjonspakkene godt
 - 👎 Det er en del parallellitet som kun oppstår i kjøretid (forgreningsprediksjon og spekulativ minneaksess)

Eksempel: Et superskalart samle- bånd med «Register renaming»





Tema 5.3: Prosessorer med høyere ytelse (Kapittel 4.11)

UT-AV-REKKEFØLGE PROSESSORER

Læringsutbytte T5.3

- Studenten skal kunne beskrive hvordan en prosessor utnytter parallellitet i tid og parallellitet i rom.
- Studenten skal kunne gjenkjenne og beskrive RAW, WAW og WAR farer og avhengighetene som skaper dem.
- Studenten skal forstå prinsippene bak hvordan «register renaming» fjerner WAW og WAR farer.
- Studenten skal kjenne til fordeler og ulemper med «static multi-issue» prosessorer.
- Studenten skal kjenne til prinsippene bak hvordan **prosessorer** som utfører **instruksjoner ut av rekkefølge («out of order»)** implementeres (**«dynamic multi-issue»**).
- Studenten skal kunne forklare begrepet **«speculation»**.

Og fra T5.2:

- Studenten skal kjenne til prinsippene bak hvordan **presise unntak** implementeres i prosessorer som utfører instruksjoner **ut-av-rekkefølge**.

5-stegs samlebandet vårt har noen utfordringer



- Vi har antatt at alle R-type instruksjoner tar like lang tid
 - ... men heltallsaddisjon krever mye enklere maskinvare enn en flyttallsaddisjon og kan dermed ta kortere tid
 - ... og klokkefrekvensen er avhengig av den tregeeste operasjonen
- Vi kan maksimalt utføre en instruksjon per klokkesykel (maksimal IPC = 1)
- Vi utfører instruksjonene i den rekkefølgen de er i programmet
 - ... og da går vi glipp av mye parallellitet

Utføringsrekkefølge

- Vi har så langt utført instruksjoner i den rekkefølgen de står i programmet
- Fundamental begrensning:
 - Når en instruksjon må stoppe, stopper alle andre instruksjoner også!
 - ... og jo lengre tid det tar å utføre en instruksjon, jo mer kunne vi ha gjort i mellomtiden
- Løsning: Ut-av-rekkefølge utføring
 - Bruker «register renaming» til å fjerne alle WAR og WAW-farer
 - Forutsetter «dynamic issue»

add p3, p2, p1

add p6, p5, p4

sub p7, p3, p6

lw p8, 0(p7)

add p11, p10, p9

add p14, p13, p12

sub p15, p11, p14

Fargene angir RAW-farer

Utføringsrekkefølge

add p3, p2, p1

add p6, p5, p4

sub p7, p3, p6

lw p8, 0(p7)

add p11, p10, p9

add p14, p13, p12

sub p15, p11, p14



Utføringsrekkefølge

Antar at maskinen kan utføre 3 instruksjoner samtidig

add	p3,	p2,	p1	add	p6,	p5,	p4	add	p11,	p10,	p9
sub	p7,	p3,	p6					add	p14,	p13,	p12
lw	p8,	0 (p7)						sub	p15,	p11,	p14



Vi har frikoblet utføringen av denne gruppen instruksjoner fra load-instruksjonen

«Data flow limit» = Det antallet instruksjoner vi kan utføre samtidig når vi kun tar hensyn til sanne dataavhengigheter

Ut-av-rekkefølge prosessorer når «data flow limit» når de har nok maskinvareressurser tilgjengelig

Eksempel: En ut-av-rekkefølge prosessor



TDT4255

Presise unntak og spekulasjon

- Vi må kunne fjerne instruksjoner fra samlebåndet helt frem til vi vet at de skal utføres:
 - Alle tidligere forgreningsinstruksjoner må være utført
 - Ingen tidligere instruksjoner kan ha generert unntak
- «Speculation»: Prosessoren utfører instruksjoner som den ikke er sikker på at den skal utføre
- «Re-Order Buffer (ROB)» og «Register Renaming» samarbeider om å implementere presise unntak
 - ROB holder styr på rekkefølgen
 - «Register Renaming» holder styr på verdiene



TDT4255



(Kapittel 2 til 4)

OPPSUMMERING: INSTRUKSJONER OG PROSESSORER

Hva kan vi nå?

Vi vet mye om instruksjoner

- Hvordan uttrykkes et høy-nivå program med instruksjoner?
- Hvorfor er instruksjonene slik som de er?

Vi kjenner til de fundamentale komponentene i prosessorer

- ALUer, registerfil, registre, kontrollenhet, etc.

Vi kan sette sammen de fundamentale komponentene til enkle prosessorer

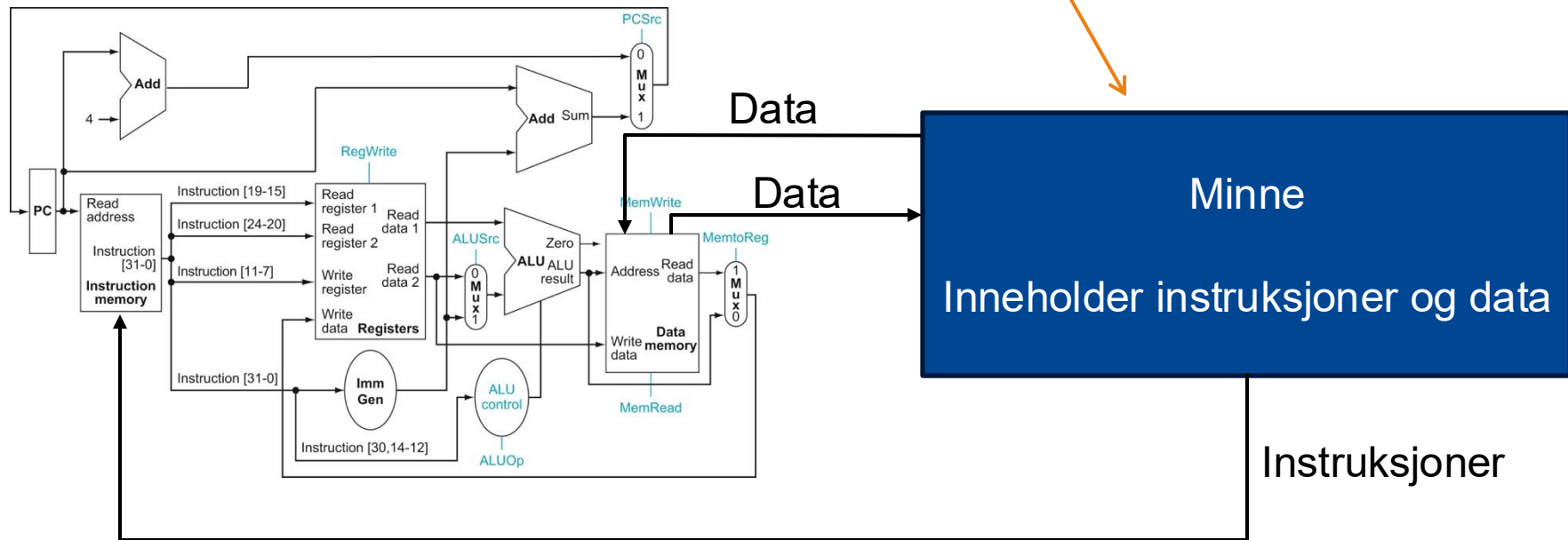
- Enkeltskykel, flersykel, og ulike samlebåndsarkitekturer

Vi kjenner til prinsippene bak moderne høy-ytelsesprosessorer

- «Data-flow execution»

Prinsippet om lagrede program

Nå skal vi finne ut hvordan denne virker



Med andre ord: Nå vet vi alt vi trenger å vite om prosessorer